

Hoja de Ruta del Proyecto "Pokefisi"

- **Fase 1: Arquitectura y Datos (El Cimiento)**
 - [] 1.1 Crear la estructura de carpetas del proyecto.
 - [] 1.2 Crear una base de datos en formato JSON para los 30 Pokémon (separando la información del código Python).
- **Fase 2: Multimedia y Assets (Lo Visual)**
 - [] 2.1 Descargar e integrar los sprites estáticos (PNG) para la lista inferior.
 - [] 2.2 Descargar e integrar los sprites animados (GIF) para la pantalla central desde Pokémon Showdown.
 - [] 2.3 Instalar y configurar la librería Pillow para manejar transparencias y redimensionar imágenes.
- **Fase 3: Refactorización y Escalabilidad (El Motor)**
 - [] 3.1 Crear un "Gestor de Ventanas" (Window Manager) para cambiar entre pantallas (Menú -> Selección -> Batalla) sin crear un desorden de ventanas.
 - [] 3.2 Separar la lógica de selección de Pokémon en un archivo logica.py y la interfaz en interfaz.py.
- **Fase 4: Pulido Profesional (La Magia)**
 - [] 4.1 Añadir imágenes de fondo (Backgrounds) de alta calidad.
 - [] 4.2 Reemplazar los marcos de Tkinter con imágenes de interfaz personalizadas (botones gráficos, paneles translúcidos).

1. Crea la estructura de carpetas

En tu computadora (dentro de

c:\Users\Yessly_Josselyn\OneDrive\Documentos\ProyectoIA_pokefisi), crea exactamente estas carpetas. Esto es un estándar en la industria:

Plaintext

```
ProyectoIA_pokefisi/
├── assets/
│   ├── backgrounds/
│   ├── sprites_mini/
│   │   (PNG)
│   └── sprites_anim/
└── data/
```

Aquí guardaremos toda la multimedia
Fondos de pantalla (ej. bg_azul.png)
Las imágenes estáticas para los botones
Los GIFs para la vista central (GIF)
Aquí guardaremos las bases de datos

```

├── pokemons.json      # Archivo con todos los stats
├── src/                # Código fuente
│   ├── main.py        # El archivo que arranca el juego
│   ├── screens.py     # Lógica de las pantallas (Azul, Blanca,
etc)
│   └── game_logic.py  # Lógica de Minimax y stats
├── requirements.txt    # Lista de librerías a instalar
├── agents.py          # Agente aleatorio Nivel 1 Y Agente Heurístico Nivel 2
└── simulación.py      # Este script enfrentará a nivel 1 vs nivel 2 en
100 combates automatizados y dirá quien gana más veces.

```

2. Crea tu Base de Datos JSON

Data/ pokemons.json

Para extraer las imágenes de Showdown que usarás luego:

- Los GIFs animados están aquí:
<https://play.pokemonshowdown.com/sprites/ani/bulbasaur.gif>
- Las imágenes miniatura estáticas están aquí:
<https://play.pokemonshowdown.com/sprites/dex/bulbasaur.png>

Automatización de descarga de imágenes

1. Instalar la librería requerida Abre tu terminal (la misma donde ejecutaste el código anterior) y escribe el siguiente comando para instalar requests:

Bash

pip install requests

2. Crea el archivo del script En la carpeta raíz de tu proyecto (junto a la carpeta data y assets), crea un archivo llamado descargar_imagenes.py.

3. Código en descargar_imagenes.py Este script lee tu archivo pokemons.json, formatea el nombre de cada Pokémon (Showdown quita los guiones, por lo que "Porygon-Z" pasa a ser "porygonz" en la URL) y descarga la miniatura y el GIF animado en las carpetas correspondientes.

4. Ejecuta el script En tu terminal, ejecuta el archivo:

Bash

python descargar_imagenes.py

En la terminal se ve como se descarga uno por uno. Si vas a las carpetas assets/sprites_mini y assets/sprites_anim.

Paso 2.3: Instalar Pillow

Tkinter por defecto es un poco limitado manejando imágenes (especialmente si queremos redimensionarlas para que se vean bien en tu interfaz HD). La industria estándar en Python para manejar imágenes es **Pillow**.

ejecuta este comando en tu terminal:

```
Bash
```

```
pip install Pillow
```

descargar_imagenes.py

```
import json
```

```
import os
```

```
import requests
```

```
import time
```

```
# Rutas de las carpetas
```

```
JSON_PATH = "data/pokemons.json"
```

```
DIR_MINI = "assets/sprites_mini"
```

```
DIR_ANIM = "assets/sprites_anim"
```

```
# Crear las carpetas si no existen
```

```
os.makedirs(DIR_MINI, exist_ok=True)
```

```
os.makedirs(DIR_ANIM, exist_ok=True)
```

```
def limpiar_nombre(nombre):
```

```
    """Formatea el nombre para que coincida con las URLs de Showdown"""
```

```
    return nombre.lower().replace("-", "").replace(" ", "").replace(".", "")
```

```
def descargar_imagen(url, ruta_destino):
```

```
    """Descarga una imagen de una URL y la guarda en la ruta indicada"""
```

```
    if os.path.exists(ruta_destino):
```

```
print(f"👉 Ya existe: {ruta_destino} (Saltando...)")  
return True
```

try:

```
respuesta = requests.get(url, stream=True)  
if respuesta.status_code == 200:  
    with open(ruta_destino, 'wb') as archivo:  
        for chunk in respuesta.iter_content(1024):  
            archivo.write(chunk)
```

```
print(f"✅ Descargado: {ruta_destino}")
```

```
return True
```

else:

```
print(f"❌ Error {respuesta.status_code} al descargar: {url}")
```

```
return False
```

except Exception as e:

```
print(f"⚠️ Excepción al descargar {url}: {e}")
```

```
return False
```

def iniciar_descarga():

```
print("Iniciando descarga de Assets desde Pokémon Showdown...")
```

Leer la base de datos

```
with open(JSON_PATH, 'r', encoding='utf-8') as f:
```

```
    pokemons = json.load(f)
```

for poke in pokemons:

```
    nombre_limpio = limpiar_nombre(poke["name"])
```

```

# URLs oficiales de Pokémon Showdown

url_mini =
f"https://play.pokemonshowdown.com/sprites/dex/{nombre_limpio}.png"

url_anim =
f"https://play.pokemonshowdown.com/sprites/ani/{nombre_limpio}.gif"


# Rutas donde se guardarán (leyendo desde el JSON)

ruta_mini = poke["img_mini"]

ruta_anim = poke["img_large_gif"]


# Descargar

descargar_imagen(url_mini, ruta_mini)

descargar_imagen(url_anim, ruta_anim)


# Pequeña pausa para no saturar el servidor de Showdown

time.sleep(0.3)


print("\n🎉 ¡Descarga completada! Revisa tus carpetas 'assets'.")


if __name__ == "__main__":

    iniciar_descarga()

```

```

game_logic.py           # Lógica de Minimax y stats
import json
import random
import math

# =====
# CLASE MOVIMIENTO
# =====
class Movimiento:
    def __init__(self, name, power, accuracy, move_type):
        self.name = name
        self.power = power
        self.accuracy = accuracy
        self.type = move_type

```

```

# =====
# CLASE POKEMON
# =====
class Pokemon:
    def __init__(self, data):
        self.id = data["id"]
        self.name = data["name"]
        self.tipo = data["tipo"]

        # Atributos Base
        self.max_hp = data["stats"]["hp"]
        self.current_hp = self.max_hp
        self.attack = data["stats"]["atk"]
        self.defense = data["stats"]["def"]
        self.speed = data["stats"]["spe"]

        # URLs de imágenes para cuando las conectemos con Tkinter
        self.img_mini = data["img_mini"]
        self.img_large_gif = data["img_large_gif"]

        # REQUISITO: Seleccionar aleatoriamente 4 movimientos de los 8
        disponibles
        self.movimientos =
self._seleccionar_movimientos(data["moves"])

    def _seleccionar_movimientos(self, moves_data):
        # random.sample toma 4 elementos únicos de la lista
        seleccionados = random.sample(moves_data, 4)
        return [Movimiento(m["name"], m["power"], m["accuracy"],
m["type"]) for m in seleccionados]

    def recibir_dano(self, cantidad):
        self.current_hp -= cantidad
        if self.current_hp < 0:
            self.current_hp = 0

    def esta_debilitado(self):
        return self.current_hp <= 0

# =====
# CLASE COMBATE Y FÓRMULA DE DAÑO (ACTUALIZADA FASE 2)
# =====
class Combate:
    def __init__(self, equipo1, equipo2, factor_k=0.1):
        self.equipo1 = equipo1
        self.equipo2 = equipo2
        self.k = factor_k
        self.turno_actual = 1

        # Inician los primeros de la lista
        self.pokemon_actuall = self.equipo1[0]
        self.pokemon_actual2 = self.equipo2[0]

    def juego_terminado(self):
        vivos_j1 = sum(1 for p in self.equipo1 if not
p.esta_debilitado())
        vivos_j2 = sum(1 for p in self.equipo2 if not
p.esta_debilitado())

        if vivos_j1 == 0:

```

```

        return 2 # Gana Jugador 2
    elif vivos_j2 == 0:
        return 1 # Gana Jugador 1
    return 0 # Sigue el juego

def calcular_dano(self, atacante, defensor, movimiento):
    if movimiento.power == 0:
        return 0
    termino1 = (atacante.attack / defensor.defense) *
movimiento.power
    termino2 = defensor.speed * self.k
    damage = termino1 - termino2
    import math
    return max(1, math.floor(damage))

def aplicar_accion(self, jugador, accion):
    """Ejecuta una acción directamente (usado cuando la IA debe
forzar un cambio)"""
    tipo, indice = accion
    if tipo == "CAMBIAR":
        if jugador == 1:
            self.pokemon_actuall = self.equipo1[indice]
        else:
            self.pokemon_actuall2 = self.equipo2[indice]

def resolver_turno(self, accion1, accion2):
    """Resuelve el turno considerando que los cambios van primero
y luego los ataques basados en velocidad"""
    tipo1, idx1 = accion1
    tipo2, idx2 = accion2

    # 1. FASE DE CAMBIOS (Los cambios siempre ocurren antes que
los ataques)
    if tipo1 == "CAMBIAR":
        self.pokemon_actuall = self.equipo1[idx1]
    if tipo2 == "CAMBIAR":
        self.pokemon_actuall2 = self.equipo2[idx2]

    # 2. FASE DE ATAQUE
    # Determinamos quién ataca primero por velocidad
    primero, segundo = 1, 2
    act1, act2 = accion1, accion2
    poke_primerio, poke_segundo = self.pokemon_actuall,
self.pokemon_actuall2

    if self.pokemon_actuall2.speed > self.pokemon_actuall.speed:
        primero, segundo = 2, 1
        act1, act2 = accion2, accion1
        poke_primerio, poke_segundo = self.pokemon_actuall2,
self.pokemon_actuall

    # Ejecuta el ataque del primero (si eligió atacar)
    if act1[0] == "ATACAR" and not poke_primerio.esta_debilitado():
        mov = poke_primerio.movimientos[act1[1]]
        dano = self.calcular_dano(poke_primerio, poke_segundo, mov)
        poke_segundo.recibir_dano(dano)

    # Ejecuta el ataque del segundo (si sobrevivió al ataque del
primero)
    if act2[0] == "ATACAR" and not poke_segundo.esta_debilitado():
        mov = poke_segundo.movimientos[act2[1]]

```

```

        dano = self.calcular_dano(poke_segundo, poke_primer, mov)
        poke_primer.recibir_dano(dano)

        self.turno_actual += 1

# =====
# FUNCIÓN UTILITARIA PARA CARGAR DATOS
# =====
def cargar_equipo_desde_json(ruta_json, ids_equipo):
    """
    Lee el JSON y convierte los IDs seleccionados en objetos Pokemon
    jugables.
    """

    with open(ruta_json, "r", encoding="utf-8") as f:
        todos_pokemons = json.load(f)

    equipo = []
    for pid in ids_equipo:
        data = next((p for p in todos_pokemons if p["id"] == pid),
None)
        if data:
            equipo.append(Pokemon(data))
    return equipo

# --- PRUEBA DE LA FASE 1 ---
if __name__ == "__main__":
    import os

    # Asegurar la ruta correcta al JSON
    ruta_datos = os.path.join(os.path.dirname(__file__), "..", "data",
"pokemons.json")

    # Simulamos que el Jugador 1 eligió a Torterra (ID 1) y el Jugador
2 a Infernape (ID 2)
    equipo_j1 = cargar_equipo_desde_json(ruta_datos, [1])
    equipo_j2 = cargar_equipo_desde_json(ruta_datos, [2])

    if equipo_j1 and equipo_j2:
        torterra = equipo_j1[0]
        infernape = equipo_j2[0]

        # Creamos el combate
        batalla = Combate(equipo_j1, equipo_j2)

        print(f"🔴 ;COMIENZA EL COMBATE!")
        print(f"{torterra.name} (HP: {torterra.current_hp}) VS
{infernape.name} (HP: {infernape.current_hp})\n")

        # Torterra elige su primer movimiento aleatorio
        ataque_torterra = torterra.movimientos[0]

        print(f"> {torterra.name} usa {ataque_torterra.name} (Poder:
{ataque_torterra.power})")

        # Usamos la fórmula
        dano = batalla.calcular_dano(torterra, infernape,
ataque_torterra)
        infernape.recibir_dano(dano)

        print(f"🔴 ;Hizo {dano} de daño!")

```

```

        print(f"❤️ HP restante de {infernape.name}:
        {infernape.current_hp}/{infernape.max_hp}")

```

```

agents.py      # Agente aletorio Nivel 1 Y Agente Heuristico Nivel 2

# Agente aletorio Nivel 1 Y Agente Heuristico Nivel 2

import random

class AgenteAleatorio:
    """Nivel 1: Selecciona acciones de forma completamente
    aleatoria."""
    def elegir_accion(self, estado_combate, es_jugador_1):
        mi_equipo = estado_combate.equipo1 if es_jugador_1 else
estado_combate.equipo2
        mi_pokemon = estado_combate.pokemon_actual1 if es_jugador_1
else estado_combate.pokemon_actual2

        opciones = []

        # 1. Puede elegir atacar (con cualquiera de sus 4 movimientos)
        for i, mov in enumerate(mi_pokemon.movimientos):
            opciones.append(("ATACAR", i))

        # 2. Puede elegir cambiar de Pokémon (si hay vivos y no es el
actual)
        for i, pok in enumerate(mi_equipo):
            if not pok.esta_debilitado() and pok != mi_pokemon:
                opciones.append(("CAMBIAR", i))

        return random.choice(opciones)

class AgenteHeuristicoBasico:
    """Nivel 2: Toma decisiones evaluando la diferencia de HP
    (Maximiza el daño)."""
    def elegir_accion(self, estado_combate, es_jugador_1):
        mi_equipo = estado_combate.equipo1 if es_jugador_1 else
estado_combate.equipo2
        mi_pokemon = estado_combate.pokemon_actual1 if es_jugador_1
else estado_combate.pokemon_actual2
        rival_pokemon = estado_combate.pokemon_actual2 if es_jugador_1
else estado_combate.pokemon_actual1

        # Si mi Pokémon actual está vivo, voy a buscar el ataque que
haga MÁS daño
        if not mi_pokemon.esta_debilitado():
            mejor_ataque = 0
            max_dano = -1

            for i, mov in enumerate(mi_pokemon.movimientos):
                # Usamos la fórmula de daño del motor para "predecir"
el daño
                dano_esperado =
estado_combate.calcular_dano(mi_pokemon, rival_pokemon, mov)
                if dano_esperado > max_dano:
                    max_dano = dano_esperado
                    mejor_ataque = i

        return ("ATACAR", mejor_ataque)

```

```

        else:
            # Si mi Pokémon se debilitó, saco al que tenga MÁS HP
            restante de mi equipo
            mejor_cambio = 0
            max_hp = -1
            for i, pok in enumerate(mi_equipo):
                if not pok.esta_debilitado() and pok.current_hp >
max_hp:
                    max_hp = pok.current_hp
                    mejor_cambio = i
            return ("CAMBIAR", mejor_cambio)

```

simulación.py #Este scrip enfrentará a nivel 1 vs nivel 2 en 100 combates automatizados y dirá quien gana más veces.

#Este scrip enfrentará a nivel 1 vs nivel 2 en 100 combates automatizados y dirá quien gana más veces.

```
from src.game_logic import Combate, cargar_equipo_desde_json
```

```
from src.agents import AgenteAleatorio, AgenteHeuristicoBasico
```

```
import random
```

```
import os
```

```
def ejecutar_combate_ia(agente1, agente2, ids_equipo1, ids_equipo2,
verbose=False):
```

```
    ruta_datos = os.path.join("data", "pokemons.json")
```

```
    equipo1 = cargar_equipo_desde_json(ruta_datos, ids_equipo1)
```

```
    equipo2 = cargar_equipo_desde_json(ruta_datos, ids_equipo2)
```

```
    batalla = Combate(equipo1, equipo2)
```

```
    while batalla.juego_terminado() == 0 and batalla.turno_actual < 150: # Límite
para evitar bucles infinitos
```

```
        # Si un Pokémon cae, se obliga al agente a cambiar
```

```
        if batalla.pokemon_actual1.esta_debilitado():
```

```
            accion1 = agente1.elegir_accion(batalla, True)
```

```
            batalla.aplicar_accion(1, accion1)
```

```

if batalla.pokemon_actual2.esta_debilitado():

    accion2 = agente2.elegir_accion(batalla, False)

    batalla.aplicar_accion(2, accion2)


# Si el juego ya terminó tras los cambios forzados, salimos
if batalla.juego_terminado() != 0:

    break


# Elegir acciones normales

accion1 = agente1.elegir_accion(batalla, True)
accion2 = agente2.elegir_accion(batalla, False)


# Resolver turno

batalla.resolver_turno(accion1, accion2)


resultado = batalla.juego_terminado()

return resultado, batalla.turno_actual


if __name__ == "__main__":

    print("🤖 Iniciando Experimento Preliminar: Nivel 1 (Aleatorio) VS Nivel 2 (Heurístico)")


    victorias_n1 = 0

    victorias_n2 = 0

    empates = 0

    turnos_totales = 0


    NUM_COMBATES = 100

```

```

agente_n1 = AgenteAleatorio()
agente_n2 = AgenteHeuristicoBasico()

for i in range(NUM_COMBATES):

    # Seleccionar 4 Pokémon al azar para cada equipo (IDs del 1 al 30)
    equipo_n1 = random.sample(range(1, 31), 4)
    equipo_n2 = random.sample(range(1, 31), 4)

    resultado, turnos = ejecutar_combate_ia(agente_n1, agente_n2, equipo_n1,
equipo_n2)

    turnos_totales += turnos

    if resultado == 1:
        victorias_n1 += 1
    elif resultado == 2:
        victorias_n2 += 1
    else:
        empates += 1

    # Progreso
    if (i+1) % 10 == 0:
        print(f"Progreso: {i+1}/{NUM_COMBATES} combates completados...")

print("\n 🏆 RESULTADOS PRELIMINARES PARA LA ENTREGA:")
print("-" * 40)
print(f"Total de Batallas: {NUM_COMBATES}")
print(f"Victorias Agente Aleatorio (Jugador 1): {victorias_n1}
({(victorias_n1/NUM_COMBATES)*100}%")

```

```

    print(f"Victorias Agente Heurístico (Jugador 2): {victorias_n2}
    ({(victorias_n2/NUM_COMBATES)*100}%)")

    print(f"Duración promedio de la partida: {turnos_totales/NUM_COMBATES:.1f}
    turnos")

```

pokemons.json

```

[
    { "id": 1, "name": "Torterra", "dificultad": "Novice", "tipo": "Planta/Tierra", "stats": {
    "hp": 95, "atk": 109, "def": 105, "spe": 56 }, "img_mini":
    "assets/sprites_mini/torterra.png", "img_large_gif":
    "assets/sprites_anim/torterra.gif" },

    { "id": 2, "name": "Infernape", "dificultad": "Expert", "tipo": "Fuego/Lucha", "stats": {
    "hp": 76, "atk": 104, "def": 71, "spe": 108 }, "img_mini":
    "assets/sprites_mini/infernape.png", "img_large_gif":
    "assets/sprites_anim/infernape.gif" },

    { "id": 3, "name": "Empoleon", "dificultad": "Intermediate", "tipo": "Agua/Acero",
    "stats": { "hp": 84, "atk": 86, "def": 88, "spe": 60 }, "img_mini":
    "assets/sprites_mini/empoleon.png", "img_large_gif":
    "assets/sprites_anim/empoleon.gif" },

    { "id": 4, "name": "Staraptor", "dificultad": "Novice", "tipo": "Normal/Volador",
    "stats": { "hp": 85, "atk": 120, "def": 70, "spe": 100 }, "img_mini":
    "assets/sprites_mini/staraptor.png", "img_large_gif":
    "assets/sprites_anim/staraptor.gif" },

    { "id": 5, "name": "Luxray", "dificultad": "Novice", "tipo": "Eléctrico", "stats": { "hp":
    80, "atk": 120, "def": 79, "spe": 70 }, "img_mini": "assets/sprites_mini/luxray.png",
    "img_large_gif": "assets/sprites_anim/luxray.gif" },

    { "id": 6, "name": "Roserade", "dificultad": "Intermediate", "tipo": "Planta/Veneno",
    "stats": { "hp": 60, "atk": 70, "def": 65, "spe": 90 }, "img_mini":
    "assets/sprites_mini/roserade.png", "img_large_gif":
    "assets/sprites_anim/roserade.gif" },

```

```
{ "id": 7, "name": "Yanmega", "dificultad": "Intermediate", "tipo": "Bicho/Volador",  
"stats": { "hp": 86, "atk": 76, "def": 86, "spe": 95 }, "img_mini":  
"assets/sprites_mini/yanmega.png", "img_large_gif":  
"assets/sprites_anim/yanmega.gif" },
```

```
{ "id": 8, "name": "Porygon-Z", "dificultad": "Expert", "tipo": "Normal", "stats": { "hp":  
85, "atk": 80, "def": 70, "spe": 90 }, "img_mini": "assets/sprites_mini/porygon-  
z.png", "img_large_gif": "assets/sprites_anim/porygon-z.gif" },
```

```
{ "id": 9, "name": "Mamoswine", "dificultad": "Novice", "tipo": "Hielo/Tierra",  
"stats": { "hp": 110, "atk": 130, "def": 80, "spe": 80 }, "img_mini":  
"assets/sprites_mini/mamoswine.png", "img_large_gif":  
"assets/sprites_anim/mamoswine.gif" },
```

```
{ "id": 10, "name": "Gastrodon", "dificultad": "Intermediate", "tipo": "Agua/Tierra",  
"stats": { "hp": 111, "atk": 83, "def": 68, "spe": 39 }, "img_mini":  
"assets/sprites_mini/gastrodon.png", "img_large_gif":  
"assets/sprites_anim/gastrodon.gif" },
```

```
{ "id": 11, "name": "Mismagius", "dificultad": "Expert", "tipo": "Fantasma", "stats": {  
"hp": 60, "atk": 60, "def": 60, "spe": 105 }, "img_mini":  
"assets/sprites_mini/mismagius.png", "img_large_gif":  
"assets/sprites_anim/mismagius.gif" },
```

```
{ "id": 12, "name": "Honchkrow", "dificultad": "Intermediate", "tipo":  
"Siniestro/Volador", "stats": { "hp": 100, "atk": 125, "def": 52, "spe": 71 }, "img_mini":  
"assets/sprites_mini/honchkrow.png", "img_large_gif":  
"assets/sprites_anim/honchkrow.gif" },
```

```
{ "id": 13, "name": "Bronzong", "dificultad": "Intermediate", "tipo":  
"Acero/Psíquico", "stats": { "hp": 67, "atk": 89, "def": 116, "spe": 33 }, "img_mini":  
"assets/sprites_mini/bronzong.png", "img_large_gif":  
"assets/sprites_anim/bronzong.gif" },
```

```
{ "id": 14, "name": "Spiritomb", "dificultad": "Expert", "tipo": "Fantasma/Siniestro",  
"stats": { "hp": 50, "atk": 92, "def": 108, "spe": 35 }, "img_mini":  
"assets/sprites_mini/spiritomb.png", "img_large_gif":  
"assets/sprites_anim/spiritomb.gif" },
```

```
{ "id": 15, "name": "Garchomp", "dificultad": "Novice", "tipo": "Dragón/Tierra",  
"stats": { "hp": 108, "atk": 130, "def": 95, "spe": 102 }, "img_mini":  
"assets/sprites_mini/garchomp.png", "img_large_gif":  
"assets/sprites_anim/garchomp.gif" },
```

```
{ "id": 16, "name": "Lucario", "dificultad": "Expert", "tipo": "Lucha/Acero", "stats": {  
"hp": 70, "atk": 110, "def": 70, "spe": 90 }, "img_mini":
```

```
"assets/sprites_mini/lucario.png", "img_large_gif":  
"assets/sprites_anim/lucario.gif" },  
  
{ "id": 17, "name": "Hippowdon", "dificultad": "Intermediate", "tipo": "Tierra",  
"stats": { "hp": 108, "atk": 112, "def": 118, "spe": 47 }, "img_mini":  
"assets/sprites_mini/hippowdon.png", "img_large_gif":  
"assets/sprites_anim/hippowdon.gif" },  
  
{ "id": 18, "name": "Drapion", "dificultad": "Intermediate", "tipo":  
"Veneno/Siniestro", "stats": { "hp": 70, "atk": 90, "def": 110, "spe": 95 }, "img_mini":  
"assets/sprites_mini/drapion.png", "img_large_gif":  
"assets/sprites_anim/drapion.gif" },  
  
{ "id": 19, "name": "Rhyperior", "dificultad": "Novice", "tipo": "Tierra/Roca", "stats": {  
"hp": 115, "atk": 140, "def": 130, "spe": 40 }, "img_mini":  
"assets/sprites_mini/rhyperior.png", "img_large_gif":  
"assets/sprites_anim/rhyperior.gif" },  
  
{ "id": 20, "name": "Electivire", "dificultad": "Intermediate", "tipo": "Eléctrico",  
"stats": { "hp": 75, "atk": 123, "def": 67, "spe": 95 }, "img_mini":  
"assets/sprites_mini/electivire.png", "img_large_gif":  
"assets/sprites_anim/electivire.gif" },  
  
{ "id": 21, "name": "Frolass", "dificultad": "Expert", "tipo": "Hielo/Fantasma",  
"stats": { "hp": 70, "atk": 80, "def": 70, "spe": 110 }, "img_mini":  
"assets/sprites_mini/frolass.png", "img_large_gif":  
"assets/sprites_anim/frolass.gif" },  
  
{ "id": 22, "name": "Weavile", "dificultad": "Expert", "tipo": "Siniestro/Hielo", "stats":  
{ "hp": 70, "atk": 120, "def": 65, "spe": 125 }, "img_mini":  
"assets/sprites_mini/weavile.png", "img_large_gif":  
"assets/sprites_anim/weavile.gif" },  
  
{ "id": 23, "name": "Togekiss", "dificultad": "Intermediate", "tipo": "Hada/Volador",  
"stats": { "hp": 85, "atk": 50, "def": 95, "spe": 80 }, "img_mini":  
"assets/sprites_mini/togekiss.png", "img_large_gif":  
"assets/sprites_anim/togekiss.gif" },  
  
{ "id": 24, "name": "Magnezone", "dificultad": "Intermediate", "tipo":  
"Eléctrico/Acero", "stats": { "hp": 70, "atk": 70, "def": 115, "spe": 60 }, "img_mini":  
"assets/sprites_mini/magnezone.png", "img_large_gif":  
"assets/sprites_anim/magnezone.gif" },  
  
{ "id": 25, "name": "Gliscor", "dificultad": "Expert", "tipo": "Tierra/Volador", "stats": {  
"hp": 75, "atk": 95, "def": 125, "spe": 95 }, "img_mini":
```

```

"assets/sprites_mini/gliscor.png", "img_large_gif": "assets/sprites_anim/gliscor.gif"
},

{ "id": 26, "name": "Gallade", "dificultad": "Intermediate", "tipo": "Psíquico/Lucha",
"stats": { "hp": 68, "atk": 125, "def": 65, "spe": 80 }, "img_mini":
"assets/sprites_mini/gallade.png", "img_large_gif":
"assets/sprites_anim/gallade.gif" },

{ "id": 27, "name": "Leafeon", "dificultad": "Novice", "tipo": "Planta", "stats": { "hp":
65, "atk": 110, "def": 130, "spe": 95 }, "img_mini":
"assets/sprites_mini/leafeon.png", "img_large_gif":
"assets/sprites_anim/leafeon.gif" },

{ "id": 28, "name": "Glaceon", "dificultad": "Intermediate", "tipo": "Hielo", "stats": {
"hp": 65, "atk": 60, "def": 110, "spe": 65 }, "img_mini":
"assets/sprites_mini/glaceon.png", "img_large_gif":
"assets/sprites_anim/glaceon.gif" },

{ "id": 29, "name": "Dusknoir", "dificultad": "Expert", "tipo": "Fantasma", "stats": {
"hp": 45, "atk": 100, "def": 135, "spe": 45 }, "img_mini":
"assets/sprites_mini/dusknoir.png", "img_large_gif":
"assets/sprites_anim/dusknoir.gif" },

{ "id": 30, "name": "Magmortar", "dificultad": "Intermediate", "tipo": "Fuego",
"stats": { "hp": 75, "atk": 95, "def": 67, "spe": 83 }, "img_mini":
"assets/sprites_mini/magmortar.png", "img_large_gif":
"assets/sprites_anim/magmortar.gif" }

]

```

```

main.py

```

```

import tkinter as tk

```

```

from tkinter import messagebox

```

```

import json

```

```

import os

```

```

from PIL import Image, ImageTk, ImageSequence

```

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

```

```

def path(*r):
    return os.path.join(BASE_DIR, "..", *r)

# =====
# DATA
# =====

def cargar():
    try:
        with open(path("data", "pokemons.json"), encoding="utf-8") as f:
            return json.load(f)
    except Exception as e:
        print(f"Error cargando JSON: {e}")
        return []

POKEMONS = cargar()

# =====
# GIF ANIMATOR
# =====

class AnimatedGIF(tk.Label):
    def __init__(self, master, gif_path, size=(300, 300), bg_color="#ffffff"):
        super().__init__(master, bg=bg_color)
        self.frames = []
        self.idx = 0

    try:
        img = Image.open(gif_path)

```

```

        for f in ImageSequence.Iterator(img):

            f = f.convert("RGBA").resize(size, Image.Resampling.LANCZOS)

            self.frames.append(ImageTk.PhotoImage(f))

except Exception as e:

    print(f"Error cargando GIF {gif_path}: {e}")


if self.frames:

    self.animate()

else:

    self.config(text="[Imagen no encontrada]", fg="black")


def animate(self):

    if self.frames:

        self.config(image=self.frames[self.idx])

        self.idx = (self.idx + 1) % len(self.frames)

        self.after(50, self.animate)


# =====

# APP PRINCIPAL

# =====

class App:

    def __init__(self, root):

        self.root = root

        self.root.title("Pokefisi Pro - Selection Screen")

        self.root.geometry("1280x720")


# --- Colores Globales Corregidos ---

self.bg_main = "#f4fcf2"    # para el fondo flotante

```

```
self.bg_panel = "#c4d6c7"

self.text_title = "#123D06"
self.text_stats = "#80917a"

self.accent_green = "#22c55e"
self.accent_black = "#050404"

self.accent_yellow = "#facc15"
self.accent_blue = "#3b82f6"

# Paleta de colores vibrantes para el Carrusel y Equipo
self.paleta_vibrante = ["#82faae", "#ff7e7e", "#ffe785", "#84d3f5", "#d483ff",
"#8279ff"]

# Asignar un color a cada Pokémon por su ID para mantener consistencia
self.poke_colors = {}
for i, p in enumerate(POKEMONS):
    self.poke_colors[p["id"]] = self.paleta_vibrante[i % len(self.paleta_vibrante)]

self.root.configure(bg=self.bg_main)

self.selected = []
self.current = POKEMONS[0]["id"] if POKEMONS else None
self.cache = {}

if not POKEMONS:
    messagebox.showerror("Error", "No se encontraron datos en
pokemons.json")
```

```

        return

    self.build_ui()

    self.update_all()

# =====

# IMG CACHE

# =====

def img(self, ruta, size, key):

    if key in self.cache:

        return self.cache[key]

    try:

        i = Image.open(path(ruta)).convert("RGBA")

        i = i.resize(size, Image.Resampling.LANCZOS)

        p = ImageTk.PhotoImage(i)

        self.cache[key] = p

        return p

    except Exception as e:

        print(f"Error imagen estática: {e}")

        return None

# =====

# EVENTOS HOVER

# =====

def bind_hover(self, widget, color_normal, color_hover):

    widget.bind("<Enter>", lambda e: widget.config(bg=color_hover))

    widget.bind("<Leave>", lambda e: widget.config(bg=color_normal))

```

```

# =====

# UI

# =====

def build_ui(self):

    # ----- IZQ (Información)

    self.left = tk.Frame(self.root, bg=self.bg_main)

    self.left.place(relx=0.05, rely=0.1, relwidth=0.25, relheight=0.6)

    self.lbl_name = tk.Label(self.left, font=("Helvetica Neue", 38, "bold"),
fg=self.text_title, bg=self.bg_main, anchor="w")

    self.lbl_name.pack(fill="x", pady=(0, 10))

    self.lbl_stats = tk.Label(self.left, font=("Courier New", 14, "bold"),
fg=self.text_stats, bg=self.bg_main, justify="left", anchor="w")

    self.lbl_stats.pack(fill="x", pady=10)

    # ----- CENTRO (Animación)

    self.center = tk.Frame(self.root, bg=self.bg_main)

    self.center.place(relx=0.35, rely=0.1, relwidth=0.30, relheight=0.55)

    self.gif = None

    self.btn_add = tk.Button(self.root, text="◆ AÑADIR AL EQUIPO ◆",
bg=self.accent_green, fg="white", font=("Arial", 14, "bold"), bd=0, cursor="hand2",
padx=20, pady=10, command=self.add)

    self.btn_add.place(relx=0.5, rely=0.68, anchor="center")

    self.bind_hover(self.btn_add, self.accent_green, "#6fdd67")

    # ----- DERECHA (Equipo Seleccionado)

```

```

self.right = tk.Frame(self.root, bg=self.bg_main, bd=0)

self.right.place(relx=0.70, rely=0.1, relwidth=0.25, relheight=0.55)

tk.Label(self.right, text="¡Tus Pokemones Elegidos!", fg=self.text_title,
bg=self.bg_main, font=("Arial", 20, "bold"), pady=15).pack()

self.team_frame = tk.Frame(self.right, bg=self.bg_main)

self.team_frame.pack(fill="both", expand=True, padx=10)

self.btn_team = tk.Button(self.root, text="VER EQUIPO (0/4)",
bg=self.accent_green, fg="white", font=("Arial", 13, "bold"), bd=0, cursor="hand2",
padx=20, pady=10, command=self.ver_equipo)

self.btn_team.place(relx=0.95, rely=0.04, anchor="ne")

self.bind_hover(self.btn_team, self.accent_green, "#4ade80")

# ----- CARRUSEL INFERIOR

self.carousel_container = tk.Frame(self.root, bg=self.bg_panel)

self.carousel_container.place(relx=0, rely=0.78, relwidth=1, relheight=0.22)

btn_izq = tk.Button(self.carousel_container, text="◀", font=("Arial", 35, "bold"),
bg=self.bg_panel, fg=self.text_title, bd=0, cursor="hand2", command=lambda:
self.canvas.xview_scroll(-2, "units"), width=3)

btn_izq.pack(side="left", fill="y")

self.bind_hover(btn_izq, self.bg_panel, "#46774b")

self.canvas = tk.Canvas(self.carousel_container, bg=self.bg_panel,
highlightthickness=0)

self.canvas.pack(side="left", fill="both", expand=True)

```

```
btn_der = tk.Button(self.carousel_container, text="►", font=("Arial", 35, "bold"),
bg=self.bg_panel, fg=self.text_title, bd=0, cursor="hand2", command=lambda:
self.canvas.xview_scroll(2, "units"), width=3)
```

```
btn_der.pack(side="right", fill="y")
```

```
self.bind_hover(btn_der, self.bg_panel, "#46774b")
```

```
self.inner = tk.Frame(self.canvas, bg=self.bg_panel)
```

```
self.canvas.create_window((0,0), window=self.inner, anchor="nw")
```

```
self.items = []
```

```
for p in POKEMONS:
```

```
    color_fondo = self.poke_colors[p["id"]]
```

```
    f = tk.Frame(self.inner, bg=color_fondo, width=100, height=130,
cursor="hand2")
```

```
    f.pack(side="left", padx=10, pady=15)
```

```
    f.pack_propagate(False)
```

```
    img = self.img(p["img_mini"], (70, 70), f"mini_{p['id']}")
```

```
    l_img = tk.Label(f, image=img, bg=color_fondo)
```

```
    l_img.pack(pady=(5,0))
```

```
    # Texto dinámico (negro si el fondo es amarillo, blanco para los demás)
```

```
    color_texto = "Green" if color_fondo == "#e8e8e8" else "white"
```

```
    l_txt = tk.Label(f, text=p["name"].upper(), fg=color_texto, bg=color_fondo,
font=("Arial", 10, "bold"))
```

```
    l_txt.pack()
```

```
    badge = tk.Label(f, bg="green", fg="white", font=("Arial", 15, "bold"))
```

```

for w in [f, l_img, l_txt]:

    w.bind("<Button-1>", lambda e, id=p["id"]: self.select(id))

    self.items.append({"id": p["id"], "frame": f, "lbls": [l_img, l_txt], "badge":
badge, "default_bg": color_fondo, "text_color": color_texto})

    self.inner.bind("<Configure>", lambda e:
self.canvas.configure(scrollregion=self.canvas.bbox("all")))

# =====

# UPDATE

# =====

def update_all(self):

    p = next(x for x in POKEMONS if x["id"] == self.current)

    self.lbl_name.config(text=p["name"].upper())

    s = p["stats"]

    stats_txt = (

        f"TIPO: {p.get('tipo', 'Normal').upper()}\n"

        f"{'-'*20}\n"

        f"HP : {s['hp']:>4}\n"

        f"ATK : {s['atk']:>4}\n"

        f"DEF : {s['def']:>4}\n"

        f"SPD : {s['spe']:>4}"

    )

    self.lbl_stats.config(text=stats_txt)

```

```

if self.gif:

    self.gif.destroy()

    self.gif = AnimatedGIF(self.center, path(p["img_large_gif"]), size=(360, 360),
bg_color=self.bg_main)

    self.gif.pack(expand=True)

self.update_team()

self.update_badges()

self.update_carousel_selection()


self.btn_team.config(text=f"VER EQUIPO ({len(self.selected)}/4)")


def update_carousel_selection(self):

    """Resalta el Pokémon seleccionado poniéndolo en color blanco"""

    for it in self.items:

        if it["id"] == self.current:

            bg_color = "#ffffff" # Blanco para el activo

            fg_color = self.text_title # Texto azul para el activo

        else:

            bg_color = it["default_bg"]

            fg_color = it["text_color"]

        it["frame"].config(bg=bg_color)

        it["lbls"][0].config(bg=bg_color) # Imagen

        it["lbls"][1].config(bg=bg_color, fg=fg_color) # Texto

# =====

```

```

def update_team(self):

    for w in self.team_frame.winfo_children():

        w.destroy()

    for i in range(4):

        if i < len(self.selected):

            p = next(x for x in POKEMONS if x["id"] == self.selected[i])

            color_poke = self.poke_colors[p["id"]]

            color_texto = "white" if color_poke == "#facc15" else "green"

            # Slot Ocupado (Usa el color vibrante del Pokémon)

            f = tk.Frame(self.team_frame, bg=color_poke, cursor="hand2")

            f.pack(fill="x", pady=6)

            tk.Label(f, text=str(i+1), bg="green", fg="white", font=("Arial", 13, "bold"),
width=3).pack(side="left", fill="y")

            img = self.img(p["img_mini"], (45, 45), f"s_{p['id']}")

            tk.Label(f, image=img, bg=color_poke).pack(side="left", padx=5)

            tk.Label(f, text=p["name"].upper(), bg=color_poke, fg=color_texto,
font=("Arial", 10, "bold")).pack(side="left")

            # Botón de eliminar

            tk.Label(f, text="✖", bg=color_poke, fg="black" if color_poke=="#09cd3d"
else "red", font=("Arial", 20)).pack(side="right", padx=10)

        for child in f.winfo_children():

            child.bind("<Button-1>", lambda e, id=p["id"]: self.remove(id))

```

```

        f.bind("<Button-1>", lambda e, id=p["id"]: self.remove(id))

    else:

        # Slot Vacío (PLOMO)

        f = tk.Frame(self.team_frame, bg=self.bg_panel)

        f.pack(fill="x", pady=6)

        tk.Label(f, text=str(i+1), bg=self.text_stats, fg="white", font=("Arial", 10,
"bold"), width=3).pack(side="left", fill="y")

        tk.Label(f, text="[ VACÍO ]", bg=self.bg_panel, fg=self.text_stats,
font=("Arial", 10), height=3).pack(side="left", padx=20)


# =====

def update_badges(self):

    for it in self.items:

        if it["id"] in self.selected:

            idx = self.selected.index(it["id"]) + 1

            it["badge"].config(text=f" {idx} ")

            it["badge"].place(relx=1.0, rely=0.0, anchor="ne")

        else:

            it["badge"].place_forget()


# =====

# ACCIONES

# =====

def select(self, pid):

    if self.current != pid:

        self.current = pid

        self.update_all()

def add(self):

```

```

if len(self.selected) >= 4:

    messagebox.showwarning("Límite de Equipo", "Ya tienes 4 Pokémon en tu
equipo. Elimina uno para añadir otro.")

    return

if self.current not in self.selected:

    self.selected.append(self.current)

    self.update_all()

else:

    messagebox.showinfo("Aviso", "Este Pokémon ya está en tu equipo.")


def remove(self, pid):

    if pid in self.selected:

        self.selected.remove(pid)

        self.update_all()


def ver_equipo(self):

    nombres = [p["name"].upper() for p in POKEMONS if p["id"] in self.selected]

    if nombres:

        msg = "Tu alineación para la batalla:\n\n" + "\n".join(f"★ {n}" for n in
nombres)

        messagebox.showinfo("Equipo Confirmado", msg)

    else:

        messagebox.showinfo("Equipo Vacío", "Aún no has seleccionado ningún
Pokémon.")


# =====

if __name__ == "__main__":

    root = tk.Tk()

```

```
App(root)
```

```
root.mainloop()
```